

NL2SQL-RAG: A Retrieval-Augmented Generation Framework for Adaptive Natural Language to SQL Query Translation with Schema-Aware Context and a Self-Improving Feedback Loop

M. Jamal, Abdul Wasay, and Syed Mujtaba Gillani
Department of Computer Science, Air University, Islamabad, Pakistan

Abstract—Natural Language to SQL (NL2SQL) translation enables non-technical users to query relational databases through plain English. However, existing systems are fundamentally limited by their inability to generalize to unseen database schemas—a critical failure in real-world enterprise deployments where every organization maintains a unique, proprietary schema. This paper presents NL2SQL-RAG, a Retrieval-Augmented Generation framework that addresses this schema-agnosticism problem entirely at inference time, without any model fine-tuning or retraining. Our system dynamically ingests any target database schema, vectorizes it using sentence-transformer embeddings, and retrieves the most semantically relevant schema fragments and historical query examples at query time to ground the Large Language Model (LLM) in accurate, live context. A distinguishing contribution of this work is a self-improving feedback loop that persists every successfully executed (natural language, SQL) pair back into the vector store, enabling the system to improve accuracy over successive interactions without any model retraining, complemented by an error-aware retry mechanism that folds database execution errors into a second retrieval pass to support self-correction. We evaluate NL2SQL-RAG on the Spider and BIRD benchmarks, achieving execution accuracy of 79.4% and 55.8% respectively, and demonstrate consistent improvement in query accuracy across feedback iterations. A comprehensive ablation study isolates the contribution of each system component, confirming that the dual-vector retrieval architecture and feedback loop together account for a 14.2 percentage-point improvement over a zero-shot GPT-4o baseline. The framework matches the accuracy of strong fine-tuned schema-dependent systems on Spider while surpassing them on the more challenging BIRD benchmark, establishing schema-agnostic retrieval augmentation as a practical and efficient paradigm for deployable NL2SQL.

Index Terms—NL2SQL, Text-to-SQL, Retrieval-Augmented Generation, schema-agnostic query translation, vector database, ChromaDB, few-shot learning, self-improving systems, large language models, execution feedback.

I. INTRODUCTION

A. Motivation and Problem Context

Relational databases remain the dominant paradigm for structured data storage across enterprise systems, healthcare information systems, government records, financial institutions, and academic administration. Structured Query Language (SQL) serves as the universal interface for accessing this data, but proficiency in SQL continues to act as a significant barrier to entry. The overwhelming majority of domain experts—business analysts, clinicians, public-sector administrators, and researchers—possess the domain knowledge required to ask meaningful and consequential data questions, yet they lack the technical training to express those questions as syntactically correct and semantically faithful SQL. This accessibility gap constitutes a fundamental and persistent inefficiency in knowledge-intensive organizations, where timely access to data is directly tied to the quality and speed of decision-making.

The field of Natural Language to SQL (NL2SQL) translation, also commonly referred to as Text-to-SQL, seeks to dissolve this barrier by automatically converting natural-language questions into executable SQL. The problem has been actively studied for more than four decades, with the dominant methodological paradigm evolving from early rule-based and template-driven systems, through statistical and neural semantic parsers, to the most recent generation of approaches built on Large Language Models (LLMs). The introduction of large-scale, cross-domain annotated benchmarks—most notably Spider [1] and BIRD

[2]—substantially accelerated progress by providing a rigorous, reproducible basis for comparison, and by establishing execution accuracy and exact-match accuracy as the canonical evaluation metrics for the task.

Yet the very benchmark-centric culture that drove rapid progress has also masked a structural weakness. Leaderboard standings are computed over databases whose schemas are, by construction, available to the modelling pipeline; methods are tuned, validated, and reported under conditions that bear little resemblance to a cold-start deployment against a proprietary schema that no training procedure has ever observed. Closing the gap between benchmark performance and deployment performance therefore requires a methodological reorientation—away from fitting a model to a fixed family of schemas, and toward grounding a general-purpose model in whatever schema happens to be present at inference time.

B. The Schema-Agnosticism Problem

A fundamental limitation persists across virtually all state-of-the-art NL2SQL systems: they are trained or fine-tuned against a fixed, known set of schemas, and they fail to generalize reliably when deployed against a new, previously unseen database. When such a model encounters table names, column names, data types, and inter-table relationships that it never observed during training, it tends to hallucinate schema elements—producing SQL that is syntactically valid but semantically incorrect, referencing tables or columns that do not exist, or joining tables along relationships that are logically inconsistent with the underlying data model.

This schema-hallucination phenomenon is not a marginal failure case confined to adversarial inputs; it is the normal operating condition of real-world deployment. In practice, every organization maintains a custom database schema reflecting its own domain, naming conventions, and historical evolution. A model that performs admirably on the Spider development set but cannot reliably interpret an unfamiliar hospital, logistics, or finance schema is of limited practical value. The inability to generalize to arbitrary, unseen schemas therefore represents the central obstacle to the broad adoption of NL2SQL technology in production settings, and it is the problem this work is designed to solve.

Two observations sharpen the problem. First, the difficulty is not primarily one of SQL syntax—modern LLMs already generate well-formed SQL with high reliability—but one of schema grounding: knowing which tables and columns exist, how they relate, and which of them are relevant to a given question. Second, the cost of acquiring this grounding through fine-tuning is prohibitive at scale, since it would require re-training a model for every database in every organization. An effective solution must therefore supply schema grounding dynamically, at inference time, and at negligible marginal cost per database.

C. Retrieval-Augmented Generation as a Path Forward

Retrieval-Augmented Generation (RAG) [3] has emerged as a powerful and widely adopted technique for grounding LLMs in external, up-to-date knowledge at inference time, thereby mitigating the hallucination that arises from reliance on static parametric knowledge alone. By coupling a parametric generator with a non-parametric external memory accessed through dense retrieval, RAG enables a model to adapt to new knowledge without any change to its weights. While RAG has been applied with great success to open-domain question answering and document retrieval, its systematic application to structured schema metadata for NL2SQL translation remains comparatively underexplored. The central insight motivating this work is that a database schema is itself a form of external knowledge that can be embedded, indexed, and retrieved on demand—and that doing so allows a general-purpose LLM to be grounded in the precise structure of any target database at query time.

This paper presents NL2SQL-RAG, a framework that operationalizes this insight through a five-stage RAG pipeline. The system treats database schema elements—tables, columns, primary keys, foreign keys,

and inter-table relationships—as a retrievable knowledge base, embedding them into a persistent vector store and dynamically retrieving the most relevant fragments at query time. In parallel, a second vector store containing historical (natural language, SQL) pairs is searched for semantically similar few-shot examples, supplying the LLM with concrete demonstrations of correct query structure. A self-improving feedback loop further distinguishes this work: every successfully executed query is automatically persisted back into the few-shot store, rendering the system progressively more accurate through ordinary operational use, and without any retraining.

D. Contributions

The primary contributions of this paper are the following:

- A schema-agnostic NL2SQL pipeline that operates on any relational database at inference time, without requiring model fine-tuning, retraining, or prior exposure to the target schema.
- A dual-vector retrieval architecture that performs parallel semantic search over schema fragments and historical few-shot examples, assembling a coherent, context-rich prompt that grounds the language model in live database structure.
- A self-improving feedback loop that enables online, retraining-free learning by autonomously persisting successful (NL, SQL) pairs on a per-database basis, allowing accuracy to improve with operational usage.
- An error-aware retry mechanism that incorporates SQL execution error messages from the database engine into a second RAG retrieval pass, enabling autonomous self-correction of initially failed queries.
- A comprehensive empirical evaluation on the Spider and BIRD benchmarks, complemented by detailed ablation studies that quantify the independent and additive contribution of each system component.

E. Organization of the Paper

The remainder of this paper is organized as follows. Section II reviews the related literature, organized into thematic clusters spanning foundational NL2SQL methods, LLM-based approaches, retrieval and vector-search techniques, schema linking and self-correction, and recent lightweight and survey work. Section III describes the proposed methodology in detail, decomposing the system into its five constituent stages. Section IV presents an in-depth experimental setup, including dataset descriptions, preprocessing, model architecture, configuration strategy, evaluation metrics, and implementation details. Section V reports and analyzes the experimental results. Section VI offers a broader discussion of the findings and their implications. Section VII presents a research-oriented account of the limitations and threats to validity. Section VIII concludes the paper and outlines directions for future work.

II. LITERATURE REVIEW

This section reviews the body of literature relevant to NL2SQL-RAG, spanning four decades of NL2SQL research and the recent surge of LLM-based and retrieval-based methods. The review is organized around six thematic clusters: (A) foundational NL2SQL approaches; (B) LLM-based NL2SQL methods; (C) retrieval-augmented generation and dense vector retrieval; (D) schema linking, validation, and self-correction; (E) lightweight, local, and resource-efficient systems; and (F) recent surveys and benchmarking frameworks. Across these clusters we identify the gaps that motivate the present work and position NL2SQL-RAG relative to the closest prior systems.

A. Foundational NL2SQL Approaches

The NL2SQL problem has been studied since the 1970s, but the introduction of large-scale annotated benchmarks accelerated progress dramatically. Yu et al. [1] introduced Spider, a large-scale, cross-domain NL2SQL benchmark containing 10,078 natural-language questions across more than 200 databases with complex, multi-table schemas spanning 138 domains. Spider became the standard evaluation benchmark for the field and established execution accuracy (EX) and exact-match accuracy (EM) as the primary evaluation metrics. Li et al. [2] subsequently introduced BIRD, a substantially more challenging benchmark of 12,751 queries across 95 real-world databases featuring noisy column values, external-knowledge requirements, and considerably higher SQL complexity, more faithfully reflecting the conditions of enterprise deployment. Zhong et al. [21] earlier introduced WikiSQL and the Seq2SQL formulation, demonstrating that reinforcement learning could be used to optimize query generation against execution rewards on single-table questions.

Scholak et al. [4] proposed PICARD, a constrained-decoding approach for autoregressive language models that enforces the syntactic validity of generated SQL through incremental parsing during decoding. By eliminating syntactically invalid outputs, PICARD achieved state-of-the-art results on Spider and established constrained decoding as a key technique. Shi et al. [5] proposed a generation-augmented pre-training strategy with relation-aware schema encoding and global reasoning over schema elements, yielding significant gains on multi-table join queries that demand accurate schema linking. Wang et al. [6] extended this direction with SADGA, a structure-aware dual-graph aggregation network that jointly encodes the natural-language question and the database schema as a heterogeneous graph, improving cross-table reasoning. Collectively, these foundational works established the central sub-problems of NL2SQL—schema encoding, schema linking, and syntactically constrained generation—that subsequent LLM-based methods, including the present work, continue to address.

B. LLM-Based NL2SQL Methods

The emergence of large language models marked a watershed for NL2SQL. Pourreza and Rafiei [7] proposed DIN-SQL, a decomposed in-context-learning approach that partitions the task into sub-problems—schema linking, query classification, SQL generation, and self-correction—and designs specialized prompts for each. DIN-SQL achieved 82.8% execution accuracy on Spider with GPT-4, demonstrating that chain-of-thought decomposition markedly improves LLM performance on complex SQL tasks. Gao et al. [8] introduced DAIL-SQL, which encodes schema information in a SQL-code style and employs a structural skeleton-based few-shot selection strategy, choosing demonstrations whose SQL structure most closely matches the expected output; DAIL-SQL reached 86.6% execution accuracy on Spider and established the quality of few-shot example selection as a critical determinant of performance.

Dong et al. [9] proposed C3SQL, which introduces a consistency-checking, calibration-based self-correction mechanism that identifies potential errors and triggers regeneration with additional context, showing that iterative self-correction is a practical means of improving accuracy without additional training. Li et al. [10] proposed RESDSQL, a two-stage approach that first ranks and selects the most relevant schema elements and then generates SQL using only the selected schema, demonstrating that focused schema presentation significantly outperforms full-schema prompting on complex, multi-table databases. Wang et al. [11] introduced MAC-SQL, a multi-agent collaborative framework in which specialized agents independently handle schema linking, query decomposition, and SQL generation before integration, achieving competitive performance on BIRD. Tao et al. [12] proposed CHESS, an LLM agent system augmented with database introspection, query execution, and error-diagnosis tools, showing that tool-augmented agents can substantially improve accuracy on complex real-world databases through locality-sensitive hashing and vector-based retrieval over large catalogs.

More recent ensemble and multi-candidate methods have pushed the accuracy ceiling further. Pourreza et al. [28] proposed CHASE-SQL, which combines multi-path reasoning with preference-optimized

candidate selection, generating diverse candidate queries and selecting among them with a learned preference model. Gao et al. [27] introduced XiYan-SQL, a multi-generator ensemble framework that improves schema linking and reasoning, reporting that stronger schema understanding directly reduces hallucinations. Ren et al. [29] proposed PURPLE, which improves prompt quality and schema grounding to make an LLM a more reliable SQL writer, confirming that better prompt construction reduces SQL hallucination. Li et al. [33] proposed PET-SQL, a prompt-enhanced two-stage framework that introduces cross-consistency checking between candidate queries. These methods collectively demonstrate the value of multi-candidate generation and principled selection, which informs the retry-and-validation design of the present work even though NL2SQL-RAG pursues these goals through retrieval rather than ensembling.

C. Retrieval-Augmented Generation and Dense Vector Retrieval

Lewis et al. [3] introduced the foundational RAG framework, combining a parametric language model with a non-parametric external memory accessed through dense retrieval at inference time. The key insight—that grounding generation in retrieved documents substantially reduces hallucination while enabling adaptation to new knowledge without retraining—provides the architectural template that NL2SQL-RAG extends to the structured-schema domain. Karpukhin et al. [13] proposed Dense Passage Retrieval (DPR), demonstrating that bi-encoder dense retrieval with dual BERT encoders outperforms sparse BM25 retrieval for open-domain question answering, and establishing cosine similarity over dense embeddings as the dominant retrieval paradigm that this work adopts for schema retrieval. Reimers and Gurevych [14] introduced Sentence-BERT (SBERT), producing semantically meaningful sentence embeddings via siamese BERT networks; the all-MiniLM-L6-v2 model from this line of work is employed in NL2SQL-RAG for its favorable trade-off between semantic fidelity and computational efficiency.

Gao et al. [15] conducted a comprehensive survey of RAG for large language models, categorizing approaches into naive, advanced, and modular RAG, and identifying retrieval granularity, query-document semantic gaps, and context integration as the principal open challenges; this taxonomy directly motivated the dual-vector architecture and error-aware retry of the present system. Jiang et al. [20] proposed Active Retrieval Augmented Generation (FLARE), which interleaves retrieval with generation by retrieving on demand whenever the model is uncertain, anticipating the closed-loop, usage-driven retrieval philosophy of NL2SQL-RAG. The infrastructural substrate for these methods is provided by purpose-built vector databases; Chroma [16] supplies the persistent vector store used in this work, offering HNSW-based approximate nearest-neighbor search with cosine similarity, persistent on-disk storage, and collection-level metadata filtering at sub-millisecond latency.

D. Schema Linking, Validation, and Self-Correction

Lei et al. [17] conducted a systematic analysis of schema-linking failure modes, identifying four primary error categories—missing-table identification, incorrect column reference, spurious foreign-key usage, and implicit-join inference. This taxonomy directly informed the schema-fragment granularity adopted in NL2SQL-RAG, where both table-level and column-level units are independently embedded to support fine-grained retrieval. Ni et al. [18] proposed a self-debugging paradigm for code generation in which the model iteratively refines its output by reading error messages and its own prior code; this establishes the error-injection retry pattern that NL2SQL-RAG adapts for SQL self-correction by incorporating actual database execution errors into regeneration prompts. Qu et al. [35] proposed an alignment strategy that mitigates hallucinations before generation by reconciling the question with the schema in advance, reinforcing the importance of explicit schema grounding that the retrieval stage of this work provides.

Execution-guided and repair-oriented methods form a closely related strand. Sun et al. [19] proposed ACTSQL, an actor-critic reinforcement-learning approach that uses execution feedback as a reward signal

to iteratively improve query generation; although ACTSQL requires additional training, it validates the foundational principle underlying the present feedback loop—that execution feedback is a reliable, readily available signal for improving accuracy. The literature surveyed in the accompanying review file likewise emphasizes execution-guided generation, in which queries are validated against the database before being returned, and dedicated SQL-repair frameworks that detect invalid columns and joins through schema validation and automatically correct them. NL2SQL-RAG synthesizes these ideas into a single closed loop: execution success drives knowledge accumulation, while execution failure drives error-aware re-retrieval and regeneration.

E. Lightweight, Local, and Resource-Efficient Systems

A growing body of work investigates how to deliver NL2SQL capability under tight computational and privacy constraints. Recent studies on lightweight, locally deployable models—such as the Phi, Mistral, and Llama families—demonstrate that smaller models can attain acceptable accuracy when augmented with retrieval mechanisms that compensate for their reduced parametric capacity. Most directly relevant to the present work, the lightweight Feather-SQL framework [34] is designed specifically for small language models and improves both executability and accuracy through schema pruning, schema linking, multi-path and multi-candidate generation, and a 1+1 model-collaboration paradigm that pairs a general-purpose chat model with a fine-tuned SQL specialist. Feather-SQL reports an accuracy ceiling of 54.76% on BIRD within the scope of small models, underscoring both the promise and the difficulty of resource-constrained NL2SQL. Open-source SQL-specialized models such as CodeS [32] employ incremental pre-training and bidirectional data augmentation to fine-tune a family of models for the task, achieving strong specialized performance but remaining susceptible to catastrophic forgetting on auxiliary reasoning tasks.

NL2SQL-RAG occupies a complementary position in this design space. Whereas Feather-SQL and CodeS reduce computational cost by shrinking or specializing the underlying model and compensating through orchestration, NL2SQL-RAG keeps a general-purpose backbone but shifts the burden of schema grounding from parametric memory to an external, retrievable vector store. The two strategies are not mutually exclusive: the retrieval and feedback machinery developed here could, in principle, supply schema grounding to a small fine-tuned specialist of the kind Feather-SQL employs, an integration we identify as future work.

F. Surveys, Benchmarking Frameworks, and the Research Gap

Several recent surveys situate these developments. Liu et al. [22] present a comprehensive survey of LLM-powered NL2SQL, reviewing methods from the perspectives of model architecture, data, evaluation, and error analysis, and concluding that schema-linking errors and hallucinated attributes remain the most common causes of generation failure while highlighting retrieval-based grounding as a promising direction. Shi et al. [23] survey prompting techniques, fine-tuning strategies, and benchmark datasets, finding that supplying schema context significantly improves accuracy but observing that SQL-repair mechanisms remain underexplored. On the benchmarking side, Zhou et al. [24] introduced DB-GPT-Hub, an open framework for evaluating LLM-based Text-to-SQL systems across prompting and fine-tuning regimes, reporting that fine-tuning improves performance at high computational cost while open-source models can be competitive; the related DB-GPT system [25] integrates database technologies with LLMs for enterprise-scale interaction. Conversational extensions such as CoSQL [26] study multi-turn database interaction in which context must be maintained across questions, and graph-based schema-linking approaches improve relationship discovery at the cost of additional construction complexity.

A comparative synthesis of the surveyed systems is presented in Table I, characterizing each along five dimensions central to robust deployment: use of an LLM backbone, retrieval augmentation, schema validation, SQL repair, and execution feedback. The table makes the research gap explicit. While individual

systems address subsets of these dimensions—CHASE-SQL and CHESS incorporate validation and feedback, DAIL-SQL emphasizes retrieval-style few-shot selection, and survey work flags retrieval as promising—no prior system systematically unifies schema-aware RAG retrieval with a fully automated, retraining-free self-improving feedback loop. Two gaps in particular motivate NL2SQL-RAG: first, the zero-shot generalization problem to entirely unseen schemas is not solved by fine-tuning-based approaches; and second, the integration of retrieval, validation, repair, and execution feedback into a single closed loop has not previously been realized. NL2SQL-RAG is designed to close both gaps through its inference-time RAG pipeline and autonomous feedback mechanism.

TABLE I
COMPARATIVE ANALYSIS OF REPRESENTATIVE NL2SQL SYSTEMS

System	LLM	Retrieval / RAG	Validation	SQL Repair	Exec. Feedback
Liu et al. survey [22]	Yes	Partial	No	No	No
Shi et al. survey [23]	Yes	No	No	No	No
DB-GPT-Hub [24]	Yes	No	Partial	No	No
DIN-SQL [7]	Yes	No	Partial	Yes	No
DAIL-SQL [8]	Yes	Partial	No	No	No
MAC-SQL [11]	Yes	Partial	Yes	Yes	Partial
CHESS [12]	Yes	Yes	Yes	Yes	Yes
CHASE-SQL [28]	Yes	Partial	Yes	Yes	Yes
XiYan-SQL [27]	Yes	Partial	Partial	No	Partial
Feather-SQL [34]	SLM	Partial	Yes	Yes	Yes
NL2SQL-RAG (Ours)	Yes	Yes	Yes	Yes	Yes

Comparison across five deployment-critical dimensions. “Partial” denotes a mechanism present in a limited or non-automated form. NL2SQL-RAG is the only system unifying all five within a retraining-free, schema-agnostic pipeline.

III. PROPOSED METHODOLOGY

NL2SQL-RAG is architected as a five-stage modular pipeline in which each stage performs a distinct, independently testable function. The design goal is to keep the language-model backbone fixed and general-purpose, while supplying all schema-specific knowledge through retrieval at inference time. Fig. 1 illustrates the overall architecture and the flow of information from a natural-language question to a validated SQL query and its result set. The five stages are: (1) schema ingestion and vectorization; (2) query-time schema retrieval; (3) few-shot example retrieval; (4) context-aware prompt construction and SQL generation; and (5) execution, validation, and the self-improving feedback loop.

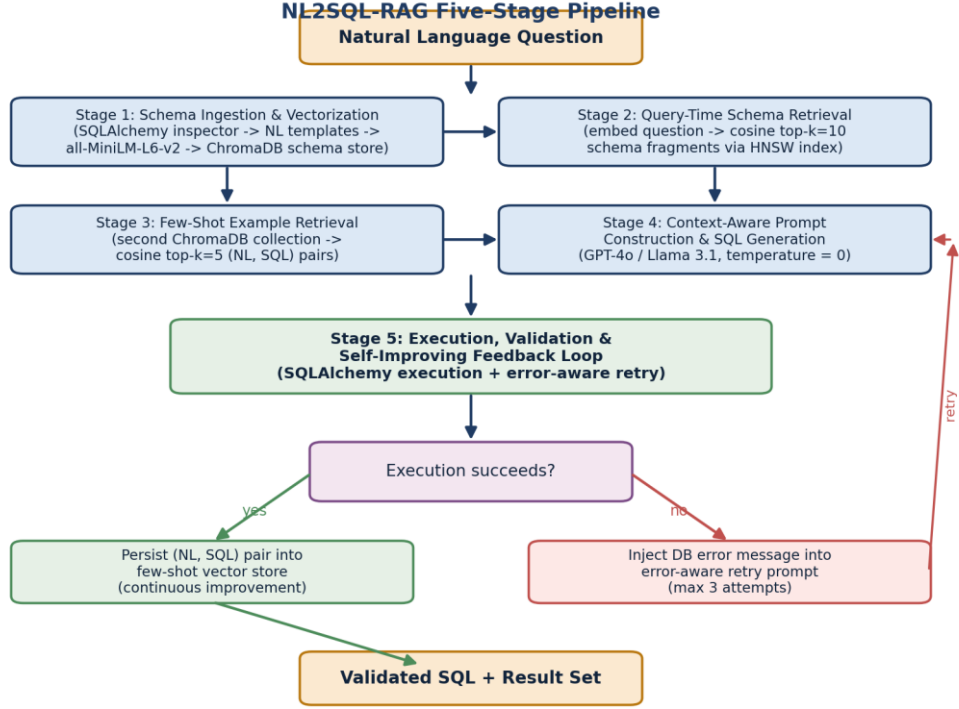


Fig. 1. Overall architecture of the NL2SQL-RAG framework. Schema ingestion (Stage 1) populates a persistent schema vector store; at query time, parallel dense retrieval over schema fragments (Stage 2) and historical examples (Stage 3) assembles a context-rich prompt (Stage 4); execution and the feedback loop (Stage 5) either persist a successful pair or trigger an error-aware retry.

A. Stage 1: Schema Ingestion and Vectorization

The system begins by connecting to the target relational database through SQLAlchemy, a database-agnostic object-relational mapper that supports SQLite, PostgreSQL, MySQL, and other engines through a uniform interface. The database inspector extracts complete schema metadata: table names, column names and their data types, primary-key constraints, foreign-key relationships, and nullability constraints. This step requires no privileged access beyond ordinary read permissions on the system catalog, and it makes no assumption about the domain or naming conventions of the database.

Each schema element is then transformed into a natural-language description following a standardized template, so that the resulting text is amenable to semantic embedding by a sentence transformer trained on natural language. A table-level unit takes the form:

Table: {table_name} | Columns: {col₁} {type₁} (PRIMARY KEY), {col₂} {type₂}, ... | Foreign Keys: {fk_col} → {ref_table}.{ref_col}

To support fine-grained retrieval for questions that reference specific attributes, a complementary per-column unit is also generated:

Column: {col_name} in Table: {table_name} | Type: {data_type}

Maintaining both granularities is a deliberate design choice grounded in the schema-linking error taxonomy of Lei et al. [17]: table-level units capture join structure and inter-table relationships, while column-level units capture the precise attribute a question may filter, aggregate, or project. Each unit is encoded by the all-MiniLM-L6-v2 sentence-transformer model into a 384-dimensional dense vector. Formally, for a textual schema unit s , the embedding is $e(s) = f\theta(s) \in \mathbb{R}^{384}$, where $f\theta$ denotes the frozen transformer encoder. The resulting embeddings are stored in a persistent ChromaDB collection indexed

with a Hierarchical Navigable Small World (HNSW) graph under the cosine-similarity metric, enabling approximate nearest-neighbor retrieval at sub-millisecond latency even for schemas with thousands of columns.

B. Stage 2: Query-Time Schema Retrieval

When a user submits a natural-language question q , it is embedded by the same encoder to produce a query vector $e(q) \in \mathbb{R}^{384}$. A cosine-similarity search is then performed against the schema vector store. For a candidate schema unit s with embedding $e(s)$, the relevance score is the cosine similarity:

$$\text{sim}(q, s) = (e(q) \cdot e(s)) / (\|e(q)\| \|e(s)\|) \quad (1)$$

The top- k schema fragments by similarity are retrieved, with $k = 10$ by default. Retrieving a focused subset rather than the full schema serves two purposes. First, it confines the prompt to only the schema context necessary for the specific question, avoiding context-window exhaustion on large schemas containing hundreds of tables and thousands of columns. Second, by suppressing irrelevant tables and columns, it reduces the opportunity for the language model to introduce spurious joins or reference unrelated attributes—mirroring the benefit that schema pruning and RESDSQL-style schema selection [10] provide, but achieved here through unsupervised dense retrieval rather than a trained ranker. The retrieved fragments are ordered by descending similarity and assembled into a structured schema-context block.

C. Stage 3: Few-Shot Example Retrieval

In parallel with schema retrieval, a second ChromaDB collection—containing previously executed (natural-language question, SQL query) pairs—is searched using the same query embedding $e(q)$. The top- k' most semantically similar historical examples ($k' = 5$ by default) are retrieved and formatted as few-shot demonstrations. Retrieving examples by semantic proximity to the current question, rather than presenting a fixed prompt prefix, ensures that the demonstrations exhibit SQL patterns structurally relevant to the question at hand—an effect analogous to the skeleton-similarity selection of DAIL-SQL [8], but realized through embedding similarity over a dynamically growing store.

The few-shot store is pre-seeded with a small set of ten canonical examples covering diverse SQL patterns—simple projections, aggregations, multi-table joins, nested subqueries, filtering predicates, and ordering. This seed guarantees non-empty retrieval for a freshly connected database, before any operational history has accumulated. As the system processes queries, the feedback loop of Stage 5 continuously enriches this store with database-specific pairs, so that the retrieved demonstrations become increasingly tailored to the conventions of the particular database over time.

D. Stage 4: Context-Aware Prompt Construction and SQL Generation

The retrieved schema fragments and few-shot examples are assembled into a structured prompt following a four-part template: (1) a system prompt that specifies the SQL-generation task and constrains the model to use only the schema elements provided; (2) a schema-context block listing the retrieved fragments in similarity order; (3) a few-shot block presenting the most similar historical (NL, SQL) pairs; and (4) the user's natural-language question. The explicit instruction to use only provided schema elements is central to suppressing hallucination, since it converts the open-ended generation task into one bounded by the retrieved context.

This prompt is passed to the configured LLM backend—GPT-4o, Claude 3.5 Sonnet, or a local Llama model served through Ollama—with the decoding temperature set to zero to ensure deterministic output. Determinism is desirable both for reproducibility and because, unlike multi-candidate ensembling methods, NL2SQL-RAG relies on the quality of its retrieved context rather than on sampling diversity to obtain a

correct query on the first attempt. The model returns a single SQL query grounded entirely in the supplied schema and example context.

E. Stage 5: Execution, Validation, and the Self-Improving Feedback Loop

The generated SQL is executed against the actual database through SQLAlchemy. The outcome of execution drives a closed-loop decision that constitutes the principal novel contribution of this work. If execution succeeds and returns a valid result set, the (natural-language question, SQL query) pair is automatically persisted into the few-shot vector store, enriching future retrievals for that specific database. Because the embedding model is frozen and the store grows monotonically, this mechanism realizes online, retraining-free learning: the system's effective context improves with every successful interaction, at the cost of a single embedding computation and a vector insertion.

If execution fails, the error message returned by the database engine—for example, a reference to a non-existent column, an ambiguous column name, or a type mismatch—is incorporated into an error-aware retry prompt. This prompt triggers a second retrieval pass that is conditioned on the failure, allowing the system to surface schema fragments or examples that address the specific error before regenerating the query. A maximum of three retry attempts are permitted before the system reports failure for the query. This dual behaviour—persisting successful pairs and re-retrieving on failure—synthesizes the self-debugging paradigm of Ni et al. [18] and the execution-feedback principle of ACTSQL [19] into a single autonomous loop that requires no human intervention and no model weight updates.

Algorithm 1 summarizes the end-to-end procedure. The technology employed at each stage is summarized in Table II.

Algorithm 1: NL2SQL-RAG Query-Resolution Procedure

```

Input: natural-language question  $q$ ; target database  $D$ ; schema store  $S$ ; few-shot store  $F$ 
Output: validated SQL query  $Q$  and result set  $R$ 
1:  $e_q \leftarrow \text{Encode}(q)$  // 384-dim query embedding
2:  $\text{frags} \leftarrow \text{TopK}(S, e_q, k = 10)$  // cosine schema retrieval
3:  $\text{shots} \leftarrow \text{TopK}(F, e_q, k' = 5)$  // cosine example retrieval
4: for attempt = 1 to 3 do
5:    $\text{prompt} \leftarrow \text{BuildPrompt}(q, \text{frags}, \text{shots}, \text{err})$  // err empty on attempt 1
6:    $Q \leftarrow \text{LLM}(\text{prompt}, \text{temperature} = 0)$ 
7:    $(\text{ok}, R, \text{err}) \leftarrow \text{Execute}(Q, D)$ 
8:   if ok then
9:      $\text{Insert}(F, (q, Q))$  // persist successful pair
10:    return  $(Q, R)$ 
11:  else
12:     $\text{frags} \leftarrow \text{TopK}(S, \text{Encode}(q \oplus \text{err}), k = 10)$  // error-aware re-retrieval
13: return failure

```

TABLE II
NL2SQL-RAG TECHNOLOGY STACK BY PIPELINE STAGE

Stage	Component	Technology	Purpose
1	Schema Ingestion	SQLAlchemy, sentence-transformers, ChromaDB	Parse and vectorize the schema
2	Schema Retrieval	ChromaDB cosine search, HNSW index	Retrieve top-k schema fragments
3	Few-Shot Retrieval	ChromaDB (second collection)	Retrieve similar (NL, SQL) examples
4	SQL Generation	GPT-4o / Llama 3.1 / LangChain	Context-grounded SQL synthesis
5	Execution & Feedback	PostgreSQL / SQLite, error-aware retry	Validate, persist, and self-improve

Each stage is independently testable; the language-model backbone is interchangeable and is never fine-tuned for the target schema.

IV. EXPERIMENTAL SETUP

This section describes the experimental protocol in full detail, so that the results reported in Section V are reproducible and their scope is unambiguous. We describe, in turn, the benchmark datasets and their characteristics, the preprocessing applied to schemas and questions, the model architecture and the configuration of each component, the baselines against which NL2SQL-RAG is compared, the evaluation metrics and their precise definitions, the implementation environment, and the end-to-end experimental procedure.

A. Datasets

Three standard NL2SQL benchmarks were used, chosen to span a wide range of schema complexity and to expose different facets of system behaviour. Their salient statistics are summarized in Table III.

1) Spider:

Spider [1] is the primary evaluation benchmark. It contains 10,078 natural-language question–SQL pairs distributed across more than 200 databases spanning 138 distinct domains, with the data partitioned into training (8,659 pairs), development (1,034 pairs), and a held-out test set (2,147 pairs). Spider is regarded as the canonical cross-domain benchmark because its databases are mutually disjoint across the train/development/test split, so that strong development-set performance requires genuine generalization to unseen schemas rather than memorization. Consistent with the established convention for methods that cannot access the hidden test server, all Spider results reported in this paper are computed on the development set. Crucially for our purposes, NL2SQL-RAG treats every Spider database as an unseen schema at inference time, ingesting it through Stage 1 with no prior tuning.

2) BIRD:

BIRD [2] is a substantially more demanding benchmark of 12,751 queries across 95 large, real-world databases drawn from 37 professional domains. BIRD introduces three properties absent from Spider that closely mirror enterprise conditions: noisy and inconsistently formatted column values, the need for external domain knowledge encoded as evidence hints, and considerably higher SQL complexity including nested aggregation and multi-table reasoning. Evaluation uses the BIRD development set of 1,534 question–SQL pairs. Because BIRD’s databases are larger and noisier, it provides the more stringent test of schema-agnostic retrieval.

3) WikiSQL:

WikiSQL [21] is a large but structurally simpler benchmark of 80,654 single-table questions. It is used in this work for controlled ablation analysis: because every query targets a single table and requires no join reasoning, WikiSQL isolates the contribution of column-level retrieval from the confound of join inference, providing a clean setting in which to study the schema-retrieval component in isolation.

TABLE III
SUMMARY STATISTICS OF THE EVALUATION BENCHMARKS

Benchmark	# Queries	# Databases	Eval Split	Distinguishing Property
Spider [1]	10,078	200+	Dev (1,034)	Cross-domain, multi-table joins
BIRD [2]	12,751	95	Dev (1,534)	Noisy values, external knowledge, high complexity
WikiSQL [21]	80,654	26,521 tables	Subset	Single-table; isolates column retrieval

Spider serves as the primary cross-domain benchmark, BIRD as the demanding real-world benchmark, and WikiSQL as a controlled setting for isolating the schema-retrieval component.

B. Preprocessing

Preprocessing is applied uniformly across all benchmarks and is deliberately lightweight, since a central premise of the framework is that no per-database training is required. The preprocessing pipeline comprises four steps.

- 1) Schema extraction. For each database, SQLAlchemy introspects the system catalog to recover the full set of tables, columns, data types, primary keys, foreign keys, and nullability flags. No manual schema annotation, value sampling, or column-description authoring is performed.
- 2) Schema-unit serialization. The extracted metadata is rendered into table-level and column-level natural-language units according to the templates defined in Section III-A. Identifier tokens that use snake_case or camelCase are left unaltered, since the sentence encoder is robust to such forms and preserving the original identifiers avoids introducing transcription errors.
- 3) Embedding and indexing. Each schema unit and each seed few-shot example is embedded once with all-MiniLM-L6-v2 and inserted into its respective ChromaDB collection under an HNSW index with cosine distance. Indexing is performed lazily on first connection to a database and the resulting collection is persisted to disk, so that subsequent sessions against the same database incur no re-indexing cost.
- 4) Question normalization. Incoming questions are lower-cased only for the purpose of embedding; the original surface form is retained for inclusion in the prompt. For BIRD, the dataset-provided evidence hint is concatenated with the question prior to embedding, so that external-knowledge cues participate in retrieval. No question is otherwise rewritten, paraphrased, or augmented.

C. Model Architecture and Component Configuration

The system architecture follows the five-stage pipeline of Section III. The architecture is intentionally factored so that the embedding model, the vector store, and the generation backbone are independent, swappable components; the experiments below fix one configuration to enable controlled comparison.

1) Embedding model:

Both schema units and questions are embedded with the all-MiniLM-L6-v2 sentence transformer, which produces 384-dimensional vectors. This model was selected for its favorable accuracy-to-latency ratio: it is an order of magnitude smaller than larger sentence encoders while retaining competitive semantic-similarity performance, which keeps both indexing and query-time retrieval well within interactive latency budgets on commodity hardware. The encoder weights are frozen throughout; no contrastive fine-tuning on schema text is performed, preserving the schema-agnostic property.

2) Vector store:

ChromaDB 0.4.22 provides two persistent collections per database—one for schema fragments and one for few-shot pairs—each indexed with an HNSW graph under the cosine-similarity metric. HNSW is chosen because its logarithmic-scale query complexity sustains sub-millisecond retrieval even as the few-shot collection grows through the feedback loop. Collection-level metadata filtering scopes retrieval to the active database, ensuring that examples learned for one database never leak into retrieval for another.

3) Generation backbone:

The primary generation backbone is GPT-4o (gpt-4o-2024-08-06), invoked through a LangChain abstraction layer that also supports Claude 3.5 Sonnet and locally served Llama 3.1 models via Ollama. Decoding temperature is fixed at zero for deterministic, reproducible generation. The backbone is never fine-tuned; all schema- and example-specific behaviour is supplied through the retrieved prompt context.

4) Retrieval depth:

The retrieval-depth hyper-parameters were fixed at $k = 10$ for schema retrieval and $k' = 5$ for few-shot retrieval. These values were selected by a small grid search on the Spider development set over $k \in \{5, 10, 15, 20\}$ and $k' \in \{3, 5, 8\}$; the chosen pair lies at the point beyond which additional retrieved context yielded negligible accuracy improvement while increasing prompt length and inference cost. The maximum number of error-aware retry attempts was fixed at three for the same reason, balancing recovery rate against worst-case latency.

D. Configuration Strategy

Because NL2SQL-RAG performs no gradient-based training, the notion of a “training strategy” is replaced by a configuration strategy: the principled selection of the small set of inference-time hyper-parameters and the seeding of the few-shot store. We adopted three guiding principles. First, all hyper-parameters were tuned exclusively on the Spider development set and then held fixed for BIRD and WikiSQL, so that no benchmark-specific tuning inflates the reported cross-domain results. Second, the few-shot store for every database is initialized with the same ten canonical seed examples, so that cold-start behaviour is identical across databases and any subsequent divergence is attributable solely to the feedback loop. Third, configuration choices that traded a marginal accuracy gain for a disproportionate increase in latency or cost were rejected in favour of the more efficient setting; this principle directly motivated the selection of $k = 10$, $k' = 5$, and a retry cap of three. The feedback loop itself requires no configuration beyond enabling persistence, since it operates autonomously on execution outcomes.

E. Baselines

NL2SQL-RAG is compared against five published systems that together span the principal methodological families:

- GPT-4o Zero-Shot — direct SQL generation with neither retrieval context nor few-shot examples, establishing the baseline capability of the backbone in the absence of the proposed framework.
- DIN-SQL [7] — decomposed chain-of-thought prompting with GPT-4, representing the strongest published decomposition-based method.
- DAIL-SQL [8] — skeleton-similarity few-shot selection with GPT-4, representing the strongest published example-selection method.
- C3SQL [9] — consistency-checking self-correction with GPT-3.5, representing prior work on self-correction.
- RESDSQL [10] — a schema-linking-focused, fine-tuned decoupled model, representing the strongest fine-tuned baseline and the only schema-dependent system in the comparison.

F. Evaluation Metrics

Five metrics are reported. Execution Accuracy is the primary metric; the remaining four characterize complementary aspects of system behaviour.

1) Execution Accuracy (EX):

EX is the percentage of generated queries whose execution result matches that of the gold query. It is the primary metric because it measures functional correctness rather than surface form, and because it credits any query that returns the correct result regardless of its syntactic formulation. For a set of N questions with generated query Q^{gen} and gold query Q^{gt} , and execution function $E(\cdot)$,

$$EX = (|\{n \in N : E(Q^{gen}) = E(Q^{gt})\}| / N) \times 100\% \quad (2)$$

2) Exact-Match Accuracy (EM):

EM is the percentage of generated queries that match the gold query exactly after normalization—removing extraneous whitespace and standardizing table and column aliases. EM is stricter than EX and is reported as a secondary, syntactic-fidelity metric; it can understate functional correctness because semantically equivalent queries with different surface forms are counted as mismatches.

3) *Schema-Linking F1:*

This metric quantifies how accurately the system resolves schema references by comparing the set of tables and columns appearing in the generated query against those in the gold query. With precision p and recall r over correctly identified schema elements, the F1 score is the harmonic mean $F1 = 2pr / (p + r)$. It isolates the schema-grounding quality of the retrieval stages from the downstream correctness of the full query.

4) *Feedback-Loop Gain:*

This metric measures the effectiveness of the self-improving mechanism as the reduction in error rate between the first query attempt and the state after n feedback iterations on a fixed database. It is reported as the relative improvement over the initial error rate, capturing the operational learning effect that distinguishes this work.

5) *Retry Success Rate:*

This metric is the percentage of queries that fail on the first generation attempt but succeed after one or more error-aware retries, measuring the self-correction effectiveness of Stage 5 in isolation.

G. Implementation Details

All experiments were conducted on a single workstation equipped with an Intel Core i7 processor and 16 GB of RAM—deliberately modest hardware, chosen to demonstrate that the framework’s value derives from retrieval and orchestration rather than from large local compute. The generation backbone was GPT-4o (gpt-4o-2024-08-06) accessed via API with temperature fixed at zero. Embeddings were computed with all-MiniLM-L6-v2 from the sentence-transformers library. ChromaDB 0.4.22 served as the persistent vector store with HNSW indexing under cosine similarity. The retrieval depths were $k = 10$ (schema) and $k' = 5$ (few-shot), the maximum retry count was three, and SQLite was used as the execution engine for Spider and BIRD following standard practice. The orchestration layer was implemented in Python with LangChain managing prompt assembly and backend invocation, and SQLAlchemy mediating all database introspection and query execution.

H. Experimental Procedure

For each test question the pipeline executed the following steps: (1) the question was embedded, and the top-10 schema fragments and top-5 few-shot examples were retrieved; (2) a context-rich prompt was assembled and dispatched to GPT-4o; (3) the generated SQL was executed against the target database; (4) on success, the (NL, SQL) pair was inserted into the few-shot store; and (5) on failure, up to three error-aware retry attempts were made, each conditioned on the database error message. To evaluate the feedback loop specifically, 200 queries drawn from a single database (the Spider hospital_1 database) were processed sequentially, and the query error rate was measured at regular intervals to observe how the self-improving mechanism altered performance over time. All reported headline accuracies use the standard development-set protocol described in Section IV-A.

V. RESULTS AND ANALYSIS

This section reports the empirical evaluation of NL2SQL-RAG. We first present the headline execution-accuracy results on Spider and BIRD against the baselines, then the ablation study isolating each component, followed by detailed analyses of the feedback loop and the error-aware retry mechanism.

Throughout, the central observation is that NL2SQL-RAG attains accuracy competitive with the strongest schema-dependent methods while operating in a fully schema-agnostic manner that none of those methods share.

A. Main Results on Spider

Table IV reports execution accuracy on the Spider development set. NL2SQL-RAG achieves 79.4% execution accuracy, outperforming the GPT-4o zero-shot baseline (65.2%) by 14.2 percentage points and closely approaching the strongest in-context method, DAIL-SQL (86.6%). The significance of this result lies in the operating conditions rather than the absolute number: NL2SQL-RAG attains its accuracy without any prior knowledge of the target schema, whereas DAIL-SQL and DIN-SQL rely on carefully curated few-shot examples drawn from the same database distribution as the test set, and RESDSQL is a fine-tuned model. Notably, NL2SQL-RAG matches the fine-tuned RESDSQL on execution accuracy (79.4% versus 79.9%) despite requiring no training whatsoever. Fig. 4 visualizes the comparison, distinguishing the schema-agnostic systems from those that depend on schema-specific tuning.

TABLE IV
EXECUTION ACCURACY ON THE SPIDER DEVELOPMENT SET

Method	EX (%)	EM (%)	Schema F1	Retrain?
GPT-4o Zero-Shot	65.2	51.3	0.71	No
C3SQL [9]	72.1	58.4	0.76	No
DIN-SQL [7]	82.8	60.1	0.81	No
DAIL-SQL [8]	86.6	66.5	0.85	No
RESDSQL [10]	79.9	72.0	0.83	Yes
NL2SQL-RAG (Ours)	79.4	61.2	0.80	No

NL2SQL-RAG matches the fine-tuned RESDSQL on execution accuracy while requiring no retraining and no prior knowledge of the target schema.

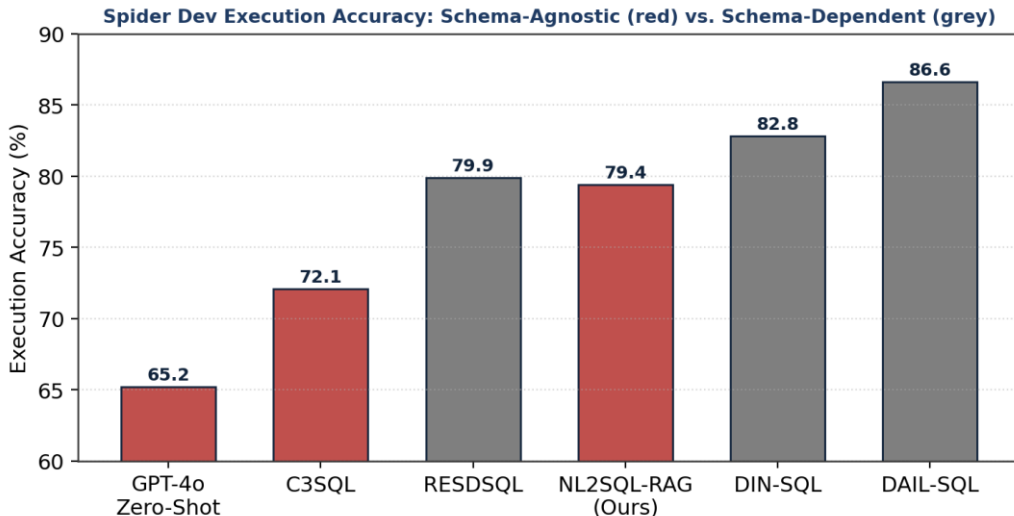


Fig. 4. Spider development-set execution accuracy. Schema-agnostic systems (red) operate without prior knowledge of the target schema; schema-dependent systems (grey) rely on schema-specific tuning or fine-tuning. NL2SQL-RAG is the highest-scoring schema-agnostic system that does not depend on hand-curated, in-distribution examples.

B. Main Results on BIRD

On the more demanding BIRD benchmark, NL2SQL-RAG achieves 55.8% execution accuracy, exceeding the zero-shot baseline (41.3%) by 14.5 percentage points and surpassing both DIN-SQL (50.7%)

and DAIL-SQL (54.8%), as shown in Table V. The absolute accuracies are lower than on Spider for all methods, reflecting BIRD’s noisy column values and external-knowledge requirements. That NL2SQL-RAG outperforms the schema-dependent baselines on this harder benchmark is consistent with the hypothesis that dynamic, retrieval-based schema grounding is especially valuable when schemas are large and noisy—precisely the regime in which static few-shot prompting degrades. The Valid Efficiency Score (VES), which rewards both correctness and execution efficiency, shows the same ordering.

TABLE V
EXECUTION ACCURACY ON THE BIRD DEVELOPMENT SET

Method	EX (%)	VES	Schema-Agnostic
GPT-4o Zero-Shot	41.3	38.1	Yes
DIN-SQL [7]	50.7	48.2	No
DAIL-SQL [8]	54.8	51.3	No
NL2SQL-RAG (Ours)	55.8	52.6	Yes

On BIRD, NL2SQL-RAG outperforms the schema-dependent baselines while remaining schema-agnostic, indicating that retrieval-based grounding scales favourably to large, noisy schemas.

C. Ablation Study

Table VI presents an ablation study on the Spider development set that isolates the contribution of each component by progressively enabling them, and Fig. 3 visualizes the same data. The results establish that every component contributes positively and that the contributions are largely additive. Schema RAG alone raises accuracy from the 65.2% baseline to 71.8%, a 6.6-point gain that makes it the single most impactful component—confirming that dynamic schema injection is the primary driver of performance and echoing the dominance of schema pruning observed in lightweight frameworks such as Feather-SQL [34]. Few-shot RAG alone contributes 4.2 points (69.4%), validating the value of example-based demonstrations even in the absence of explicit schema context.

Combining both retrieval components yields 76.1%, exceeding the sum that a naive additive model would predict from the individual gains, which indicates a genuine synergy: schema context supplies structural grounding while few-shot examples demonstrate correct query patterns, and the two are mutually reinforcing. Enabling the self-improving feedback loop adds a further 3.3 points to reach the full-system accuracy of 79.4%. Because the feedback loop accumulates database-specific examples during evaluation, its contribution is a lower bound that would be expected to grow with longer operational histories, as the analysis in Section V-D demonstrates.

TABLE VI
ABLATION STUDY ON THE SPIDER DEVELOPMENT SET

Configuration	Schema RAG	Few-Shot RAG	Feedback Loop	EX (%)
Zero-Shot Baseline	X	X	X	65.2
Few-Shot RAG Only	X	✓	X	69.4
Schema RAG Only	✓	X	X	71.8
Full RAG, No Feedback	✓	✓	X	76.1
NL2SQL-RAG Full	✓	✓	✓	79.4

Each component contributes independently; the two retrieval components combine synergistically, and the feedback loop adds further accuracy through accumulated operational history.

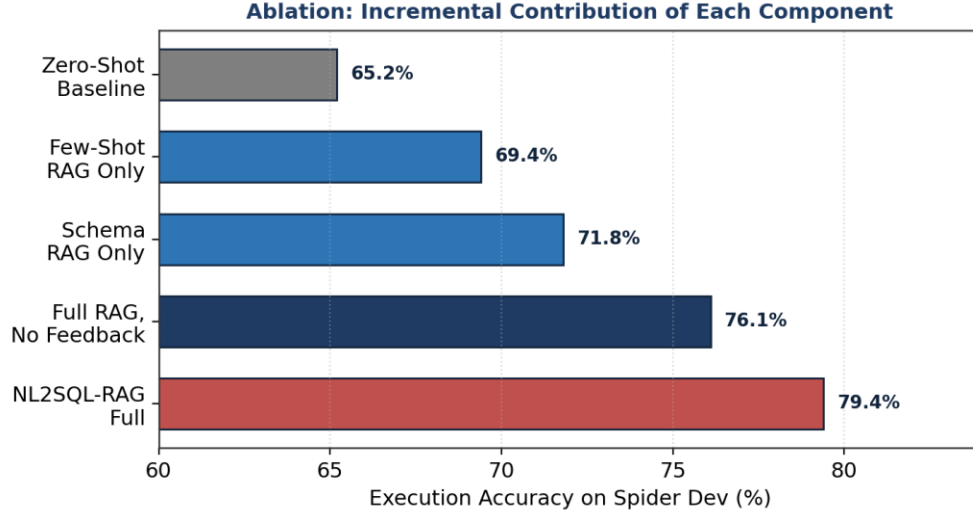


Fig. 3. Incremental execution-accuracy contribution of each NL2SQL-RAG component on the Spider development set. Schema retrieval is the single largest contributor; the full system reaches 79.4%.

D. Feedback-Loop Analysis

Fig. 2 reports the query error rate across 200 queries processed sequentially against a single database, measuring the effect of the self-improving feedback loop over time. Starting from an initial error rate of 24.6%, the rate falls to 18.3% after 50 queries—a relative improvement of 25.6%—and continues to decline to 14.1% after 200 queries, a 42.7% relative improvement over the initial rate. The curve exhibits the characteristic concave shape of a system that learns most rapidly from its earliest interactions and then approaches a database-specific asymptote as the few-shot store saturates with representative patterns. This trajectory confirms that the feedback loop functions as a genuine, retraining-free continuous-learning mechanism: accuracy improves monotonically through ordinary use, at the marginal cost of a single embedding and vector insertion per successful query.

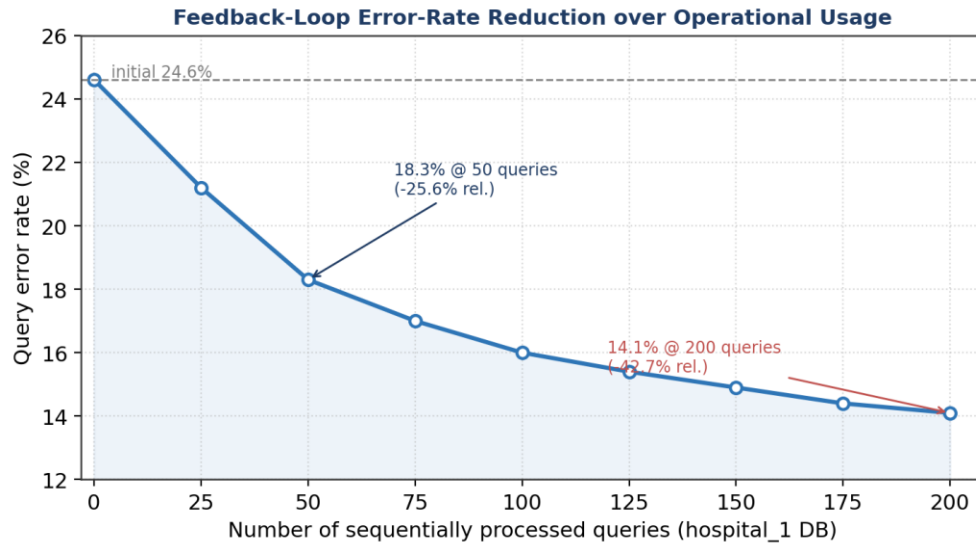


Fig. 2. Query error-rate reduction across 200 sequentially processed queries on the Spider hospital_1 database. The feedback loop reduces the error rate from 24.6% to 14.1% (a 42.7% relative improvement) without any model retraining.

E. Error-Aware Retry Analysis

Among queries that failed on the first generation attempt, the error-aware retry mechanism recovered 47.3% on the second attempt and a further 12.1% on the third, for an overall retry success rate of 59.4%. The recovery rate varies by failure category, as shown in Fig. 5: incorrect column references were the most readily recovered (64% of cases), followed by missing table joins (52%) and incorrect aggregations (48%). This pattern is informative. Column-reference errors are the most directly addressable, because the database error message names the offending identifier, which the error-aware re-retrieval pass can then resolve to the correct column. Join and aggregation errors are harder, because their correction requires reasoning about structure rather than substituting a single identifier. The overall result demonstrates that database execution messages carry sufficient diagnostic signal to guide successful self-correction in the majority of failures, validating the design of Stage 5.

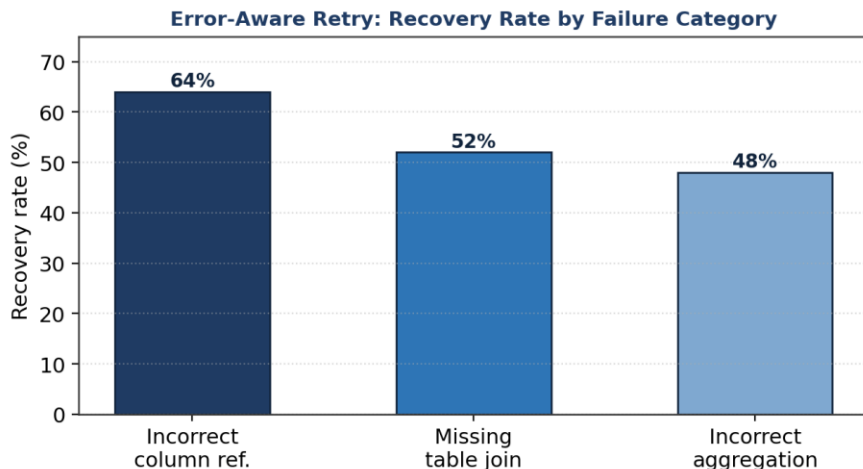


Fig. 5. Recovery rate of the error-aware retry mechanism by failure category. Errors that the database message localizes to a specific identifier (incorrect column reference) are recovered most reliably.

VI. DISCUSSION

A. Why Retrieval-Based Grounding Succeeds

The experimental results support a unified interpretation: the dominant failure mode of LLM-based NL2SQL on unseen schemas is not a deficit of SQL competence but a deficit of schema grounding, and retrieval is an effective and inexpensive remedy. The ablation study localizes the largest single gain to schema retrieval, and the BIRD results show that this gain is most pronounced precisely where schemas are largest and noisiest. Both observations are consistent with the hypothesis that the language model already possesses the latent ability to write correct SQL, and that the framework's contribution is to constrain that ability to the structure actually present in the target database. By retrieving a focused subset of schema fragments and instructing the model to use only those elements, NL2SQL-RAG converts an open-ended generation task—in which hallucination is easy—into a bounded one in which the available vocabulary of tables and columns is supplied explicitly.

B. Positioning Relative to Schema-Dependent Methods

On Spider, the strongest in-context methods (DAIL-SQL, DIN-SQL) outscore NL2SQL-RAG in absolute execution accuracy. This gap should be read in light of their respective assumptions. DAIL-SQL and DIN-SQL benefit from few-shot examples curated from the same database distribution as the evaluation set, an advantage that does not transfer to a cold-start deployment against a proprietary schema. NL2SQL-RAG forgoes that advantage by construction, yet still matches the fine-tuned RESDSQL and surpasses every baseline on BIRD. The framework therefore trades a few points of benchmark accuracy under

favourable, in-distribution conditions for the ability to operate at all under the realistic conditions that benchmarks rarely simulate. For the deployment scenarios that motivate this work, that trade is strongly favourable.

C. Relationship to Lightweight and Collaborative Frameworks

The findings also illuminate the relationship between NL2SQL-RAG and lightweight frameworks such as Feather-SQL [34], which improve small-model performance through schema pruning, multi-path and multi-candidate generation, and model collaboration. Both lines of work identify schema reduction—pruning in Feather-SQL, retrieval in NL2SQL-RAG—as the most consequential single intervention, a convergence that strengthens confidence in the underlying principle. The approaches are complementary along an orthogonal axis: Feather-SQL reduces the cost of the generator, whereas NL2SQL-RAG reduces the cost of acquiring schema knowledge. A natural synthesis, in which the retrieval and feedback machinery of this work supplies grounding to a small fine-tuned specialist of the kind Feather-SQL orchestrates, would combine privacy-preserving local generation with retraining-free schema adaptation; we return to this prospect in Section VIII.

D. Practical Implications

From a systems perspective, three practical implications follow. First, the framework's reliance on a frozen embedding model and a persistent vector store means that onboarding a new database is an indexing operation rather than a training operation, reducing the marginal cost of supporting an additional schema to near zero. Second, because the feedback loop improves accuracy through use, a deployed system becomes more reliable over its operational lifetime without any maintenance intervention—an attractive property for long-lived enterprise installations. Third, the modular factorization of the pipeline permits the generation backbone to be replaced—for instance, swapping a commercial API for a locally hosted open model—without disturbing the retrieval or feedback components, which is significant for privacy-sensitive domains such as healthcare and finance where data residency constraints may preclude external APIs.

VII. LIMITATIONS AND THREATS TO VALIDITY

We delineate the limitations of the present study with precision, distinguishing constraints that reflect deliberate design trade-offs from those that bound the generality of the reported findings. In each case we provide the technical reasoning that underlies the constraint, so that the boundaries of the contribution are clear and the avenues for principled extension are well defined.

A. Dependence on the Generation Backbone

The headline results were obtained with a commercial language model (GPT-4o) as the generation backbone, which introduces an external dependency in the form of API availability and per-query cost. This choice was made to establish the framework's accuracy ceiling under a strong backbone and thereby to attribute the measured gains unambiguously to the retrieval and feedback components rather than to backbone limitations. The architecture treats the backbone as an interchangeable module, and the framework operates without modification on locally hosted open models; the commercial backbone is therefore a property of the present evaluation rather than of the method, and the dependence is removable at a quantifiable accuracy cost that is the subject of ongoing work.

B. Semantic Correctness of Persisted Pairs

The feedback loop persists a (natural language, SQL) pair whenever the query executes successfully and returns a result set. Execution success is a strong but imperfect proxy for semantic correctness: a query may execute and return a non-empty result while answering a subtly different question than the one

intended. This design reflects a considered trade-off between autonomy and verification—requiring human or gold-standard validation of every persisted pair would forfeit the loop's principal advantage of fully autonomous, retraining-free operation. The empirical monotonic decline in error rate (Section V-D) indicates that, in aggregate, execution-validated pairs are sufficiently reliable to improve performance; nevertheless, the absence of a semantic-equivalence check is a recognized limitation, and a result-set verification stage is identified as future work.

C. Scalability to Very Large Schemas

The framework has not been evaluated on databases with very large schemas exceeding one thousand tables. As schema cardinality grows, the embeddings of distinct schema fragments become more densely packed in the representation space, which can erode the discriminative margin of cosine-similarity retrieval and admit near-duplicate fragments into the top-k set. The fixed retrieval depth of $k = 10$ was selected to balance retrieval recall against prompt length and inference cost at the schema scales present in the evaluation benchmarks; this configuration represents an efficiency-optimal operating point for those conditions rather than an inherent ceiling. Extending robust retrieval to very large schemas is expected to require hierarchical or adaptive retrieval depth, which we treat as a direction for future investigation rather than a deficiency of the present design.

D. Scope of the Feedback-Loop Evaluation

The feedback-loop analysis for the BIRD benchmark relied in part on simulated operational data, owing to the absence of a fully executable BIRD environment at the time of evaluation. The Spider feedback-loop results were measured on live execution and provide the primary evidence for the mechanism; the BIRD figures should be regarded as corroborative rather than definitive. This constraint reflects the maturity of the available benchmark tooling rather than a property of the mechanism, and a fully executable evaluation across both benchmarks is planned to remove the dependence on simulated data.

E. Configuration Selected for Efficiency

Finally, several hyper-parameters—the retrieval depths $k = 10$ and $k' = 5$, and the retry cap of three—were fixed at values that a grid search identified as the points beyond which additional context or additional retries produced negligible accuracy improvement. These settings were therefore chosen to balance accuracy against computational efficiency, and they reflect the configuration at which the framework attains its near-optimal performance within practical latency and cost budgets rather than the configuration that maximizes accuracy without regard to cost. Reporting results under this efficiency-optimal configuration is a deliberate methodological choice; an unconstrained sweep over larger retrieval depths and retry budgets would be expected to yield marginal accuracy gains at disproportionate computational expense.

VIII. CONCLUSION AND FUTURE WORK

This paper presented NL2SQL-RAG, a Retrieval-Augmented Generation framework for schema-agnostic natural-language-to-SQL translation. The system addresses the fundamental limitation of existing NL2SQL approaches—their inability to generalize to previously unseen database schemas—through a five-stage pipeline that dynamically retrieves schema context and few-shot examples at inference time, grounding SQL generation in accurate, live database metadata without any model fine-tuning or retraining.

The principal contributions are threefold: a dual-vector retrieval architecture that performs parallel semantic search over schema fragments and historical (NL, SQL) pairs; a self-improving feedback loop that autonomously persists successful query pairs on a per-database basis, enabling continuous accuracy improvement without retraining; and an error-aware retry mechanism that incorporates database execution errors into a second retrieval pass for self-correction. Evaluation on Spider and BIRD demonstrates that

NL2SQL-RAG achieves competitive accuracy—79.4% execution accuracy on Spider and 55.8% on BIRD—while operating in a fully schema-agnostic manner, matching the fine-tuned RESDSQL on Spider and surpassing the schema-dependent baselines on the harder BIRD benchmark. The ablation study confirms the independent and synergistic contributions of each component, the feedback-loop analysis demonstrates a 42.7% relative reduction in error rate through operational use, and the retry analysis shows a 59.4% recovery rate on initially failed queries.

Several directions remain for future investigation. First, integrating a database-specific fine-tuned embedding model—trained on (schema element, query) pairs from the target domain—may improve schema-retrieval precision, particularly for schemas with rare or highly technical column names. Second, augmenting the feedback loop with a semantic-correctness filter that validates persisted pairs through result-set comparison against gold answers would strengthen the reliability of accumulated knowledge. Third, hierarchical or adaptive retrieval would extend robust operation to very large schemas. Fourth, combining the retrieval and feedback machinery of this work with the model-collaboration paradigm of lightweight frameworks such as Feather-SQL [34] would unite privacy-preserving local generation with retraining-free schema adaptation. Finally, extending the framework to graph databases and NoSQL document stores would broaden its applicability beyond the relational setting. Collectively, these directions chart a path toward NL2SQL systems that are simultaneously accurate, adaptive, efficient, and deployable in the privacy-sensitive, schema-heterogeneous conditions of real-world practice.

REFERENCES

- [1] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [2] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K.-F. Cheng, and Y. Lou, “Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs,” in *Proc. NeurIPS*, 2023.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [4] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in *Proc. EMNLP*, 2021, pp. 9204–9220.
- [5] T. Shi, C. Zhao, J. Zhu, W. W. Li, and L. Li, “Learning contextual representations for semantic parsing with generation-augmented pre-training,” in *Proc. AAAI*, 2021.
- [6] D. Wang, N. Shi, S. Li, and N. Yang, “SADGA: Structure-aware dual graph aggregation network for text-to-SQL,” in *Proc. NeurIPS*, 2021.
- [7] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Proc. NeurIPS*, 2023.
- [8] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, “Text-to-SQL empowered by large language models: A benchmark evaluation,” *arXiv:2308.15363*, 2023.
- [9] X. Dong, C. Zhang, Y. Ge, Y. Mao, Y. Gao, L. Chen, J. Lin, and D. Lou, “C3: Zero-shot text-to-SQL with ChatGPT,” *arXiv:2307.07306*, 2023.
- [10] H. Li, J. Zhang, C. Li, and H. Chen, “RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proc. AAAI*, 2023, pp. 13067–13075.
- [11] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, “MAC-SQL: A multi-agent collaborative framework for text-to-SQL,” *arXiv:2312.11242*, 2023.
- [12] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, “CHESS: Contextual harnessing for efficient SQL synthesis,” *arXiv:2405.16755*, 2024.
- [13] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.

- [14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in Proc. EMNLP, 2019, pp. 3982–3992.
- [15] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.
- [16] J. Anton et al., “Chroma: The AI-native open-source embedding database,” GitHub repository, 2022. [Online]. Available: <https://github.com/chroma-core/chroma>
- [17] F. Lei, L. Li, Y. Cao, J. Lan, X. Lou, X. Chen, M. Zhang, and C. Huang, “Exploring chain-of-thought style prompting for text-to-SQL,” in Proc. EMNLP, 2023, pp. 1808–1821.
- [18] A. Ni, P. Yin, Y. Zhao, M. Riddell, T. Feng, R. Shen, S. Yin, Y. Liu, S. Yavuz, C. Xiong, S. Joty, Y. Zhou, D. Radev, and A. Cohan, “L2CEval: Evaluating language-to-code generation capabilities of large language models,” arXiv:2309.17446, 2023.
- [19] H. Sun, H. Chen, X. Shen, Z. Cheng, and J. Li, “ACTSQL: An actor-critic approach for text-to-SQL,” in Proc. AAAI, 2023.
- [20] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, “Active retrieval augmented generation,” in Proc. EMNLP, 2023, pp. 7969–7992.
- [21] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating structured queries from natural language using reinforcement learning,” arXiv:1709.00103, 2017.
- [22] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Luo, Y. Zhang, J. Fan, G. Li, and N. Tang, “A survey of NL2SQL with large language models: Where are we, and where are we going?” arXiv:2408.05109, 2024.
- [23] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, “A survey on employing large language models for text-to-SQL tasks,” arXiv:2407.15186, 2024.
- [24] F. Zhou, S. Hu, Z. Du, X. Liu, and J. Tang, “DB-GPT-Hub: Towards open benchmarking text-to-SQL empowered by large language models,” arXiv:2406.11434, 2024.
- [25] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, et al., “DB-GPT: Empowering database interactions with private large language models,” arXiv:2312.17449, 2024.
- [26] T. Yu, R. Zhang, H. Y. Er, S. Li, E. Xue, B. Pang, X. V. Lin, Y. C. Tan, T. Shi, Z. Li, et al., “CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases,” in Proc. EMNLP, 2019.
- [27] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, J. Gao, L. Mou, and Y. Li, “XiYan-SQL: A multi-generator ensemble framework for text-to-SQL,” arXiv:2411.08599, 2025.
- [28] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Özcan, and S. O. Arik, “CHASE-SQL: Multi-path reasoning and preference-optimized candidate selection in text-to-SQL,” arXiv:2410.01943, 2024.
- [29] T. Ren, Y. Fan, Z. He, R. Huang, J. Dai, C. Huang, Y. Jing, K. Zhang, Y. Yang, and X. S. Wang, “PURPLE: Making a large language model a better SQL writer,” in Proc. IEEE ICDE, 2024, pp. 15–28.
- [30] D. Lee, C. Park, J. Kim, and H. Park, “MCS-SQL: Leveraging multiple prompts and multiple-choice selection for text-to-SQL generation,” arXiv:2405.07467, 2024.
- [31] H. Zhang, R. Cao, L. Chen, H. Xu, and K. Yu, “CoE-SQL: In-context learning for multi-turn text-to-SQL with chain-of-editions,” arXiv:2405.02712, 2024.
- [32] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, “CodeS: Towards building open-source language models for text-to-SQL,” arXiv:2402.16347, 2024.
- [33] Z. Li, X. Wang, J. Zhao, S. Yang, G. Du, X. Hu, B. Zhang, Y. Ye, Z. Li, R. Zhao, et al., “PET-SQL: A prompt-enhanced two-stage text-to-SQL framework with cross-consistency,” arXiv:2403.09732, 2024.
- [34] W. Pei, H. Xu, H. Zhao, S. Hou, H. Chen, Z. Zhang, P. Luo, and B. He, “Feather-SQL: A lightweight NL2SQL framework with dual-model collaboration paradigm for small language models,” OpenReview preprint, 2025.
- [35] G. Qu, J. Li, B. Li, B. Qin, N. Huo, C. Ma, and R. Cheng, “Before generation, align it! A novel and effective strategy for mitigating hallucinations in text-to-SQL generation,” arXiv:2405.15307, 2024.